Exploring Performance Optimizations in Fluid Simulations Using Vectorization, JIT, and GPU Acceleration in Python

Aditya Melinkeri

February 11, 2026

Abstract

This report presents an overview of my efforts to optimize the performance of a realistic fluid simulation based on the paper by Müller, Charypar, and Gross (2003) on fluid simulations for virtual applications. The project investigates performance scaling and computational bottlenecks in this particle-based fluid solver and attempts to resolve them using NumPy vectorization, JIT compilation, and GPU acceleration via Taichi. Experiments analyze runtime performance across varying particle counts and implementation strategies to evaluate real-time feasibility of SPH simulations on consumer hardware. This report does not contain the exact algorithmic implementation and instead contains an overview of the optimizations employed and the observed improvements on performance

1 Introduction

Smoothed Particle Hydrodynamics (SPH) is a mesh-free Lagrangian method for simulating fluid flow. Particle-particle interactions are modelled using smoothing kernels or "Spheres of Influence", enabling simulation of complex free-surface dynamics. However, naive implementations scale poorly with particle count, making performance optimization essential for real-time or large-scale simulations.

2 Background

The choice to use Python for this project was mostly arbitrary, however the availability of highly optimised modules for scientific computation such as Numpy, Numba and Taichi simplified the coding process immensely

3 A Brief Overview of the SPH Method

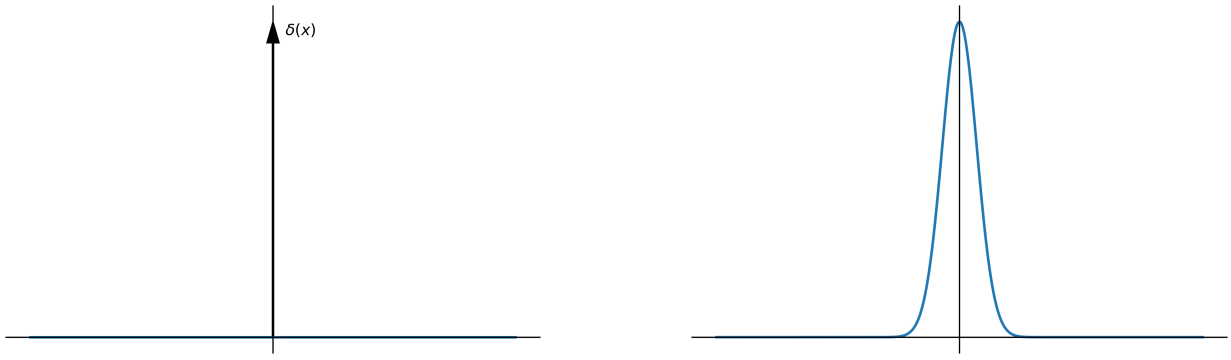
3.1 Discretizing Navier-Stokes

The governing equation of SPH is that any scalar field A can be sampled using the Dirac Delta

$$\mathbf{A(r)} = \int \boldsymbol{A(r')\delta(r - r')dr}$$

However, we can approximate the Dirac Delta function using a smoothing function(See Fig.1)

$$\mathbf{A(r)} = \int \boldsymbol{A(r')W(r - r')dr} \tag{1}$$



(a) Ideal Dirac delta

(b) Kernel approximation

Figure 1: Comparison of the Dirac delta distribution and the approximation made by the "Smoothing Function". This function drops down to zero beyond a finite distance from the origin known as the "smoothing radius" **h**.

Now, discretizing the continuous fluid medium into distinct particles, assuming a volumetric density ρ(in the case of 2D simulations ρ refers to the areal density, summing up over all particles we get

$$A_i(r) = \sum_j A_j(r') \frac{m_j}{\rho_j} W(r_j - r_i, h) \tag{2}$$

Substituting ρ_i we get the equation for the density field

$$\rho_i = \sum_j m_j W(r_{ij}, h) \tag{3}$$

The incompressible Navier Stokes equation is as follows

$$\rho \frac{\partial \mathbf{v}}{\partial t} = -\nabla p + \mu \nabla^2 \mathbf{v} + \mathbf{f}$$

We can find the divergence of scalar fields in our model by finding ∇A

$$\nabla A(\mathbf{r}) = \frac{\partial}{\partial \mathbf{r}} \int A(\mathbf{r}') W(\mathbf{r} - \mathbf{r}', h)\, d\mathbf{r}' \approx \sum_{b=1}^{N_{\text{neigh}}} m_b \frac{A_b}{\rho_b} \nabla W(\mathbf{r} - \mathbf{r}_b, h)$$

Hence can conclude the equation for the pressure force

$$\mathbf{f}_i^{\text{pressure}} = -\sum_{j \neq i} m_j \frac{p_i + p_j}{2\rho_j} \nabla W_{\text{pressure}}(\mathbf{r}_i - \mathbf{r}_j, h) \tag{4}$$

Note that average pressure P_{ij} is substituted in of P_i to maintain action-reaction symmetry

We define an equation of state from the density field using a target equilibrium density ρ_{target} and a constant labelled the "pressure multiplier" k

$$P_i = k(\rho_i - \rho_{target}) \tag{5}$$

3.2 Choice of Kernel Functions

The Poly6 kernel is used for density computation(substitute for W) due to its smooth profile and well-behaved gradient near particle centers:

$$W_{\text{poly6}}(r,h) = \begin{cases} \frac{315}{64\pi h^9}(h^2 - r^2)^3, & 0 \le r \le h \\ 0, & r > h \end{cases}$$

The Spiky kernel is employed for pressure force calculations(substitute for ∇W) because its gradient remains well-defined and avoids numerical instability when particles are in close proximity:

$$W_{\text{spiky}}(r,h) = \begin{cases} \frac{15}{\pi h^6}(h - r)^3, & 0 \le r \le h \\ 0, & r > h \end{cases}$$

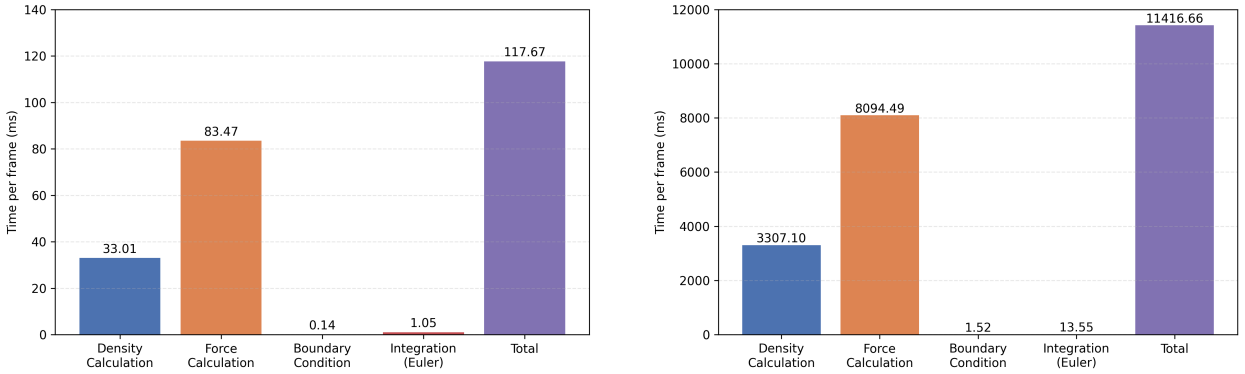
3.3 Simulation Loop

1. Compute particle densities using Eq. (3) by summing contributions from neighbouring particles **within the smoothing radius**.
2. Evaluate particle pressures from the computed density field using the equation of state given in Eq. (5).
3. Compute pressure forces by evaluating pressure gradients using Eq. (4).
4. Compute viscous forces based on velocity differences between neighbouring particles.
5. Apply external forces such as gravity where applicable.
6. Integrate accelerations to update particle velocities and positions using LeapFrog integration.
7. Enforce boundary conditions and resolve particle collisions with domain boundaries.

4 Levels of Performance Upgrade

4.1 Naive Python

This version of the code uses nested Python for loops to compute interparticle interactions for densities and pressure gradients, along with similar iterations for actions such as applying the equation of state, resolving boundary conditions, etc. The overall time complexity of this approach is $O(N^2)$.



(a) Particle Count: 100 (8.5 FPS)

(b) Particle Count: 1000 (0.009 FPS)

Figure 2: Sectionwise breakdown of computational time for Naive Python

4.2 Spatial Hashing

Calculating pairwise interaction for N particles results in ${}^{N}C_2$ expensive distance calculations which are the cause of the $O(N^2)$ complexity. The majority of these calculations are redundant as all particles outside a range have zero influence on the ith particle. Instead we divide 3D space into a volumetric grid with cells of size $h \times h \times h$.

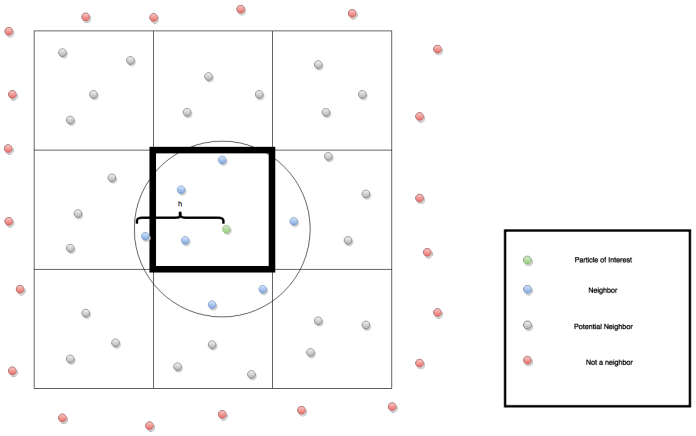
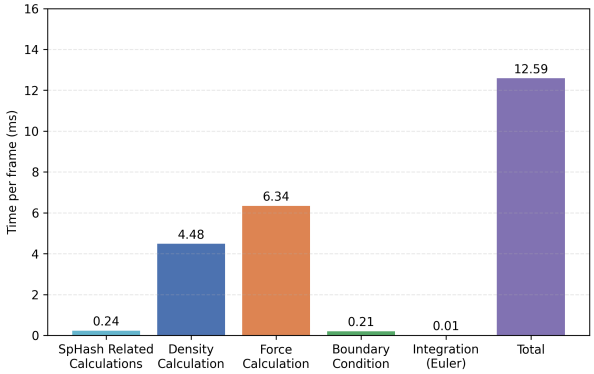
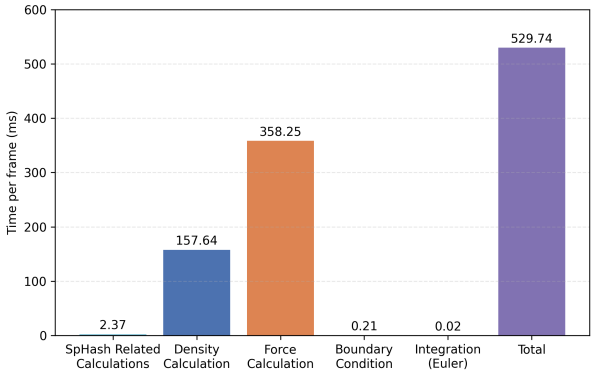


Figure 3: Spatial hashing neighbourhood search. Only particles within adjacent grid cells are considered for interaction, reducing average complexity to $O(N)$.

For a particle i, the only particles under consideration are those lying in the 3×3 grid centered at the cell containing i. This reduces the overall time complexity of the simulation to approximately $O(N)$ on average.



(a) Particle Count: 100 (79 FPS)



(b) Particle Count: 1000 (1.9 FPS)

Figure 4: Sectionwise breakdown of computational time for Spatial Hashing integrated simulation

5 Vectorization Using NumPy

As is evident from the data obtained for the naive and spatial hashing implementations, the dominant computational cost arises from repeated pairwise evaluations of kernel functions during density and pressure force computation. Even after reducing the number of interacting particle pairs using spatial hashing, the simulation still spends the majority of its runtime inside nested loops performing distance calculations and kernel point evaluations. This sort of repetitive calculation being the bottleneck pushes us towards the use of vectorization techniques to further optimize performance.

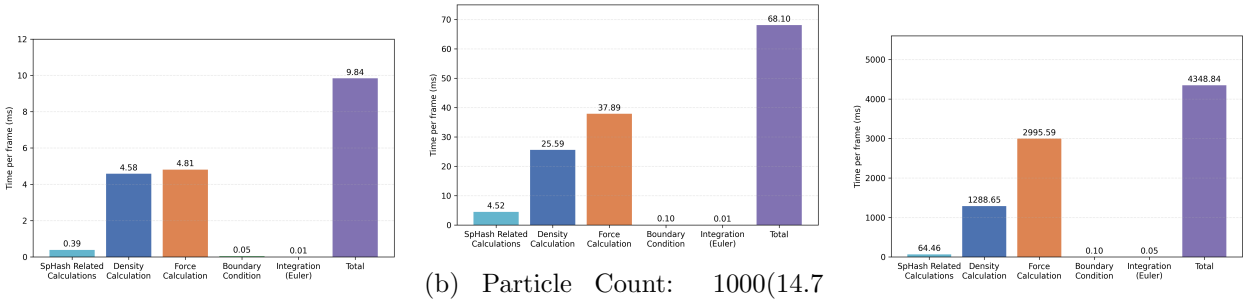
By restructuring the solver to operate on large global matrices containing particle attributes such as positions, velocities, densities, and accelerations, it becomes possible to compute interactions between many particles simultaneously rather than iterating through particle objects individually. Pairwise separation vectors between sets of particles are computed using "broadcasted"a subtraction between position matrices, producing distance matrices for entire neighborhoods in a single operation. These distance matrices are then used to evaluate smoothing kernels and their gradients in a fully vectorized manner. Consequently, core computations such as density estimation, kernel evaluation, and pressure force accumulation are performed on entire cells at once instead of through nested Python loops.

This approach leverages highly optimized low-level numerical routines implemented in C and Fortran within the NumPy module, which internally utilise SIMD vector instructions and BLAS-level optimizations to accelerate array operations. In addition to benefiting from hardware-level parallelism, this restructuring also eliminates the overhead associated with repeatedly passing particle objects into functions and accessing their attributes. Particle properties are stored in contiguous global arrays, enabling faster memory access, which is necessary for bulky computations associated with simulations. We observe a substantial reduction in per frame processing time as well as overhead, resulting in a significant improvement in overall simulation performance.

5.1 JIT Optimization

Some performance-critical routines such as the retrieval of particles in a certain neighborhood are still left using nested for loops after vectorization that remain inefficient when interpreted in pure Python due to interpreter overhead. By applying JIT compilation in nopython mode, these routines

are translated into optimized machine code at runtime, enabling execution at near C-like speeds. The speedup is relatively small at smaller counts(˜100) but becomes significant as particle counts rise

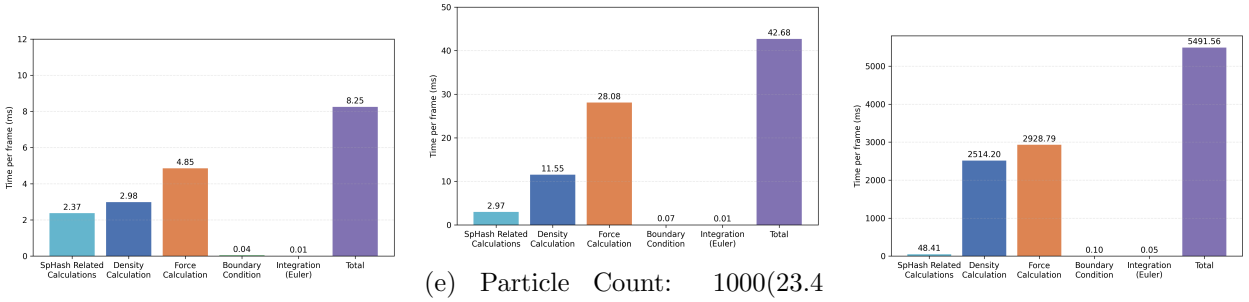


(a) Particle Count: 100(102 FPS)

(b) Particle Count: 1000(14.7 FPS)

(c) Particle Count: 10k(0.23 FPS)

Vectorised NumPy Implementation



(d) Particle Count: 100(121 FPS)

(e) Particle Count: 1000(23.4 FPS)

(f) Particle Count: 10k(0.18 FPS)

Vectorised + JIT Implementation

Figure 5: Section-wise frame time distribution across particle counts for vectorised and JIT-accelerated implementations.

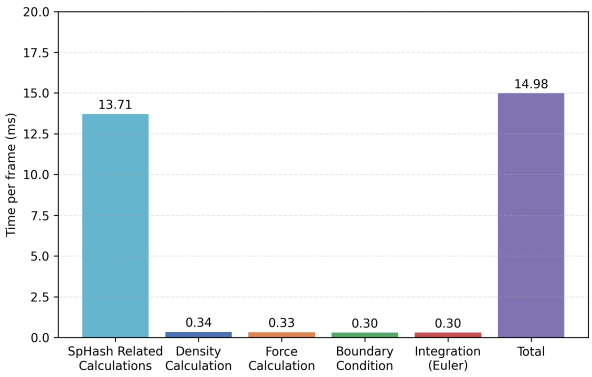
Note that as the particle count grows larger(10k and over), the density segment actually grows larger. At this stage I am unable to comment on these results, but I suspect it has to do with pure Python memory allocation being more efficient than Numba at larger scales

6 GPU Acceleration with Taichi

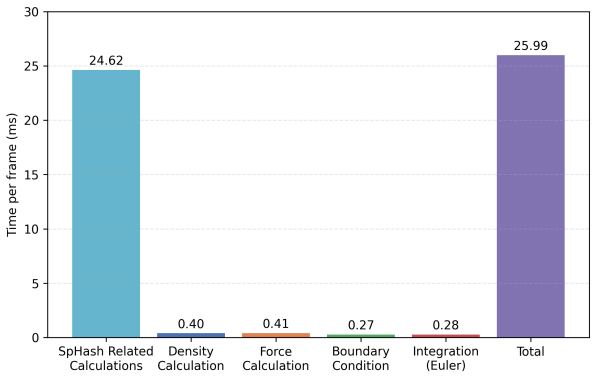
Fluid simulations naturally require that the same algorithmic steps be executed for all particles, making them prime candidates for parallel computing. Modern GPUs are designed for massive parallelism, containing thousands of lightweight cores capable of executing identical operations across large datasets simultaneously. The final stage of optimisation involves transitioning the simulation from CPU-bound execution to GPU-accelerated computation.

In this implementation, we utilize the Taichi framework in Python to compile the density and force calculation functions into CUDA kernels or GPU compute shaders(depending on backend). The kernel launches an independent parallel thread for each iteration of the top level loop in the function, most often corresponding to a single particle's compute loop, allowing for all the calculations to run simultaneously. By offloading the most computationally intensive SPH operations

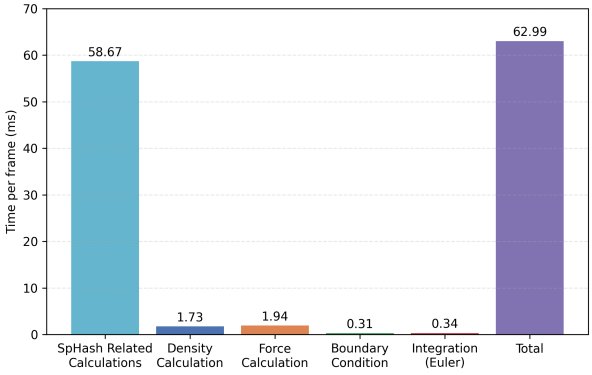
to massively parallel hardware, this approach eliminates the serial bottlenecks of CPU execution allowing for blazingly fast processing speeds, blowing past previous benchmarks



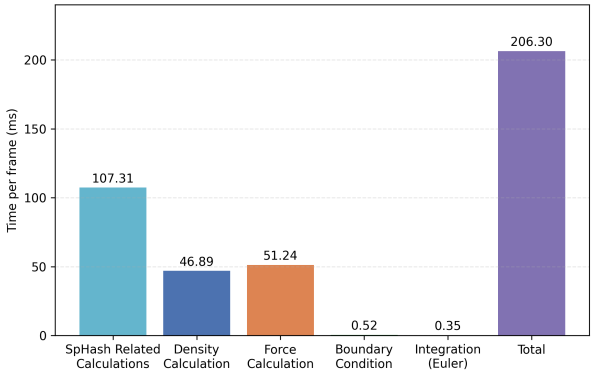
(a) Particle Count: 100 (66.7 FPS)



(b) Particle Count: 1000 (38.5 FPS)



(c) Particle Count: 10k (15.9 FPS)



(d) Particle Count: 100k (4.8 FPS)

Figure 6: Sectionwise computational time distribution for Taichi GPU implementation across increasing particle counts.

Note that the bottleneck in this version of the code lies in the spatial hashing segment, particularly due to the reliance on Taichi's builtin parallel sort algorithm. A different implementation that employs atomic bins over a sorting based approach could massively raise the ceiling for performance

7 Results

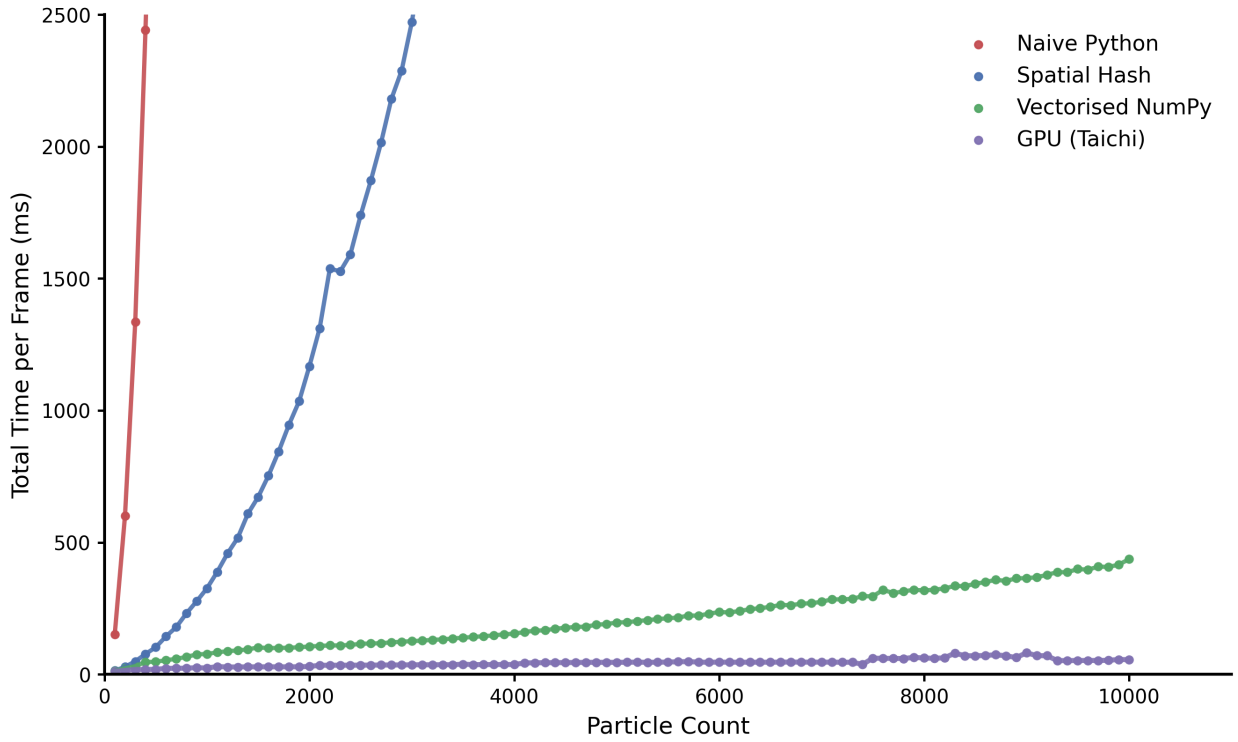


Figure 7: Comparing how each discussed algorithm scales with simulation size. All benchmarks were conducted on a consumer laptop i7 CPU GTX 1650 Ti GPU 4GB VRAM

Note, our SpHash implementation scales with $O(N)log(N)$ rather than $O(N)$ due to the design choice of implementing sorting algorithms over atomic bins. Both methods have their advantages, however that discussion is currently out of the scope of this report

8 Results and Discussion

The scaling comparison across all implementations highlights the dramatic impact of algorithmic and architectural optimisations on performance. The naive implementation exhibits clear $O(N^2)$ behaviour, quickly becoming impractical beyond a few hundred particles due to the quadratic growth in pairwise interaction computations. Even modest increases in particle count result in extreme growth in frame time, rendering real-time simulation infeasible for these implementations

The introduction of spatial hashing significantly reduces unnecessary interaction checks by limiting neighbour searches to adjacent grid cells. This reduces the average computational complexity to $O(N)log(N)$ and yields substantial performance improvements, particularly at intermediate particle counts. However, while spatial hashing certainly improves scalability, the simulation remains limited by Python loop overhead and repeated kernel evaluations, still unviable for most applications

Vectorisation restructures the computations entirely, operating on large contiguous arrays rather than individual particle objects and bringing about the first significant performance boost, bringing the simulation into the realm of feasibility at moderate scales. However this still pales in comparison

to the Taichi implementation. The use of compute shaders drastically drops frame time and more importantly, scalability, holding steady at incredibly large simulation sizes with only a 5x increase in per-frame processing time between 100 and 10k particles and 15x between 100 and 100k, as compared to the vectorised simulation with a 50x increase from 100 to 10k and a near 500x increase from 100 to 100k

9 Future Work

A yet unexplored optimisation is the concurrent use of both CPU and GPU resources. Many preprocessing steps required for neighbor search and data organisation do not strictly depend on the completion of GPU kernel execution and could therefore be executed in parallel on the CPU. For instance, while the GPU evaluates density and pressure forces for the current frame, the CPU could simultaneously compute spatial hash keys, update neighbor indexing structures, or prepare sorted particle buffers for the next timestep. This overlap could reduce idle time between kernel launches and improve overall throughput. However, exact predictions on the comparison between pure GPU computation and CPU/GPU split will certainly vary based on the user's hardware, making commenting on its feasibility difficult until further research has been completed.